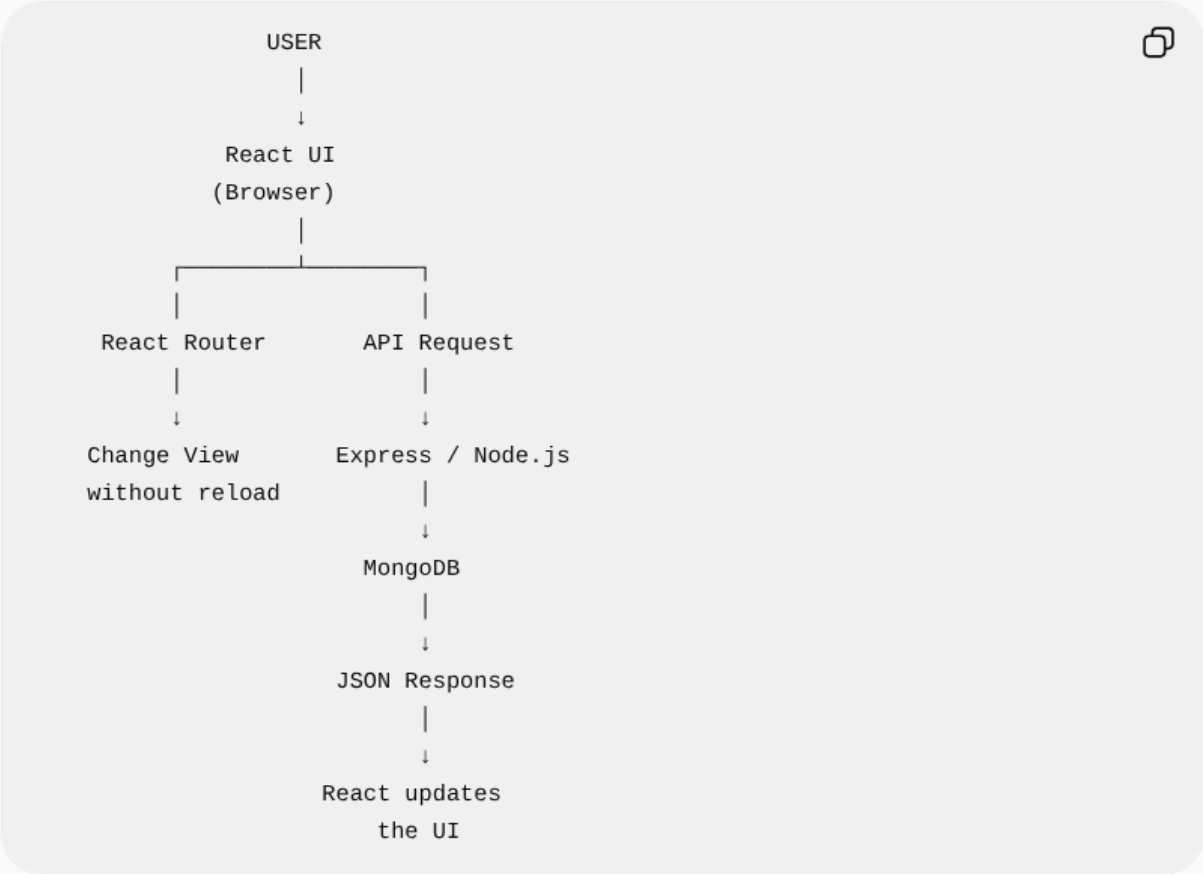
Angular MSE 1 Important

1. Explain the architecture of a Single-Page Application (SPA). How does it differ from a traditional multi-page web application?

A **Single-Page Application (SPA)** is a web application that loads a single main HTML page and **dynamically updates only the required part of the page** when the user interacts with it. It does not reload the complete web page for every navigation.

In the MERN stack, **React** is commonly used to build the SPA frontend. React runs in the browser, requests data from the API server, and dynamically renders the UI.

SPA Architecture



How SPA Works

1. Initial Request:
The browser first requests the application from the server. The server sends the HTML, CSS, and JavaScript required for the application.

2. React Loads:
The React JavaScript application starts running in the browser.

3. User Interaction:
When the user clicks a button or navigates to another section, React handles the interaction on the client side.

4. API Request:
If data is required, React sends a request to the backend API. The server returns the required data, usually in **JSON format**, rather than sending an entirely new HTML page.

5. UI Update:
React updates only the required part of the page. The complete page is not reloaded.

Example

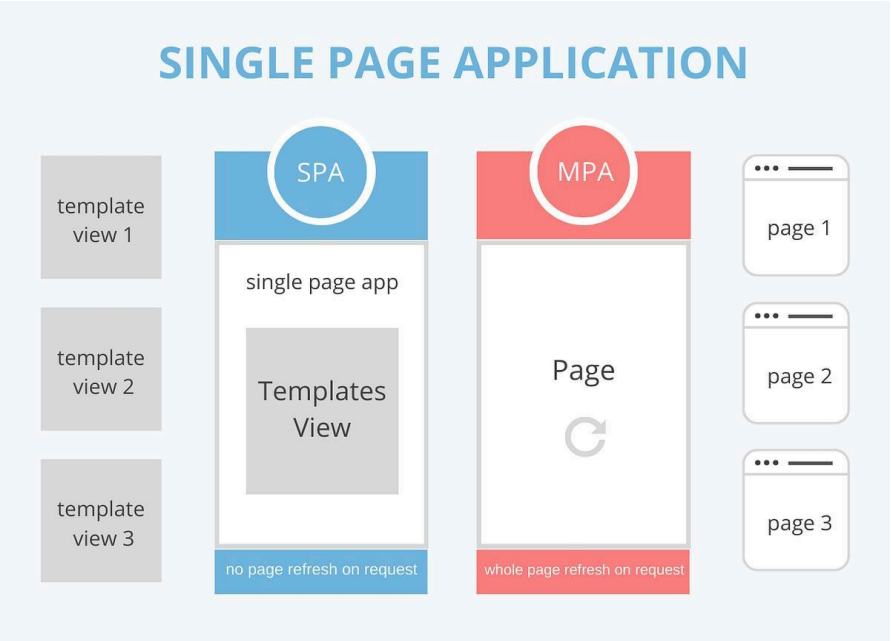
Consider an e-commerce SPA:

```
Home → Products → Cart → Profile
          |
          ↓
     Same HTML Page
          |
          ↓
    React changes only
    the required component
```

When the user moves from **Home** → **Products**, React replaces the relevant UI component instead of requesting a completely new HTML page.

Difference Between SPA and Traditional Multi-Page Application

A traditional **Multi-Page Application (MPA)** has multiple HTML pages. Every time the user navigates to another page, the browser requests a new document from the server and reloads the page.

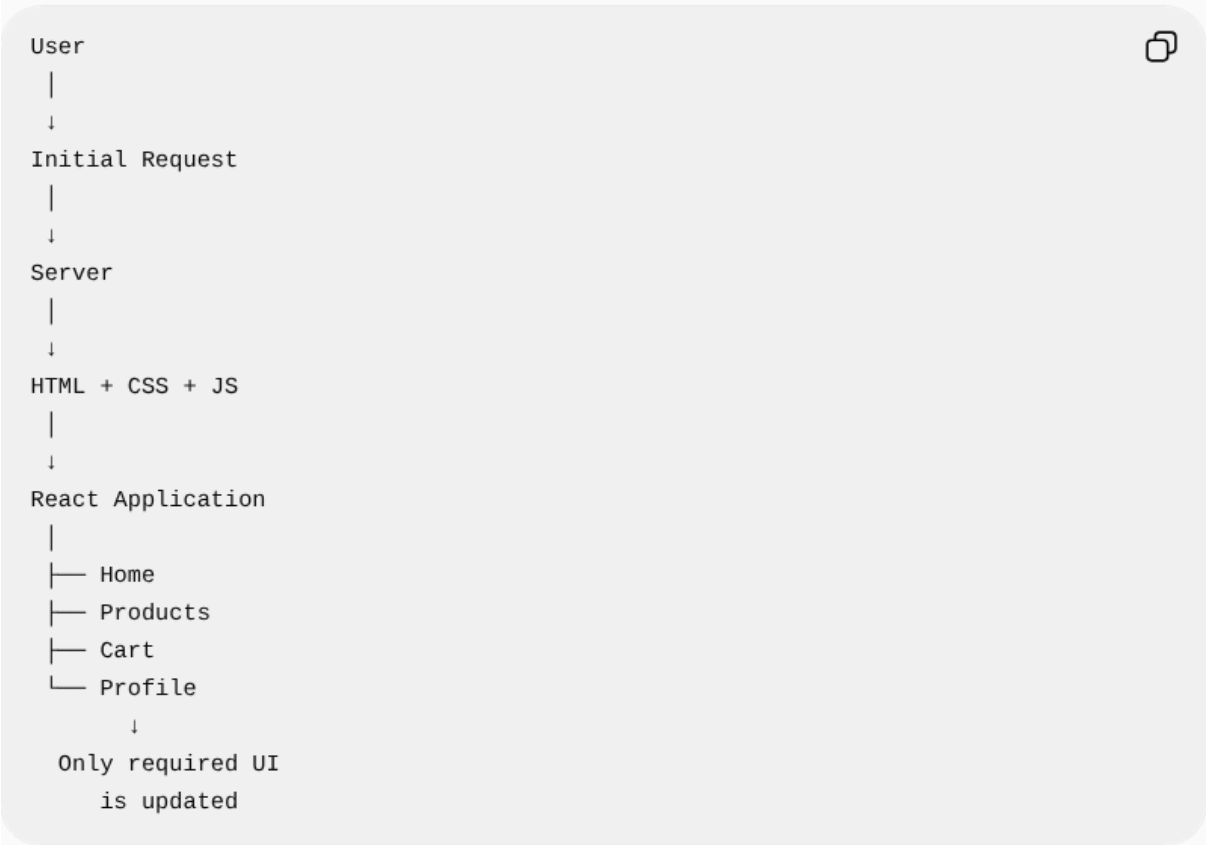


Traditional MPA

```
User
 |
 ↓
Request Home
 |
 ↓
Server
 |
 ↓
home.html
 |
 ↓
Browser

User clicks Products
 |
 ↓
New Request
 |
 ↓
Server
 |
 ↓
products.html
 |
 ↓
Full Page Reload
```

SPA



Comparison

Feature	Traditional MPA	SPA
Pages	Multiple HTML pages	Mainly one HTML page
Navigation	New page requested	Client-side navigation
Page Reload	Happens on navigation	No full-page reload
Data Requests	Often returns complete HTML	Usually returns data such as JSON
UI Update	Server sends new page	React updates required portion
User Experience	More page refreshes	Smooth and interactive
Initial Load	Generally loads current page resources	Initial load can be larger

Example	Traditional server-rendered website	React SPA

The course material specifically contrasts MPAs and SPAs: traditional applications make a new document request on each navigation, while SPAs make the initial document request and subsequently fetch data without reloading the page.

Advantages of SPA

- **Faster page updates**
- Smooth user experience
- No repeated full-page reloads
- Suitable for highly interactive applications
- Client-side routing can provide app-like navigation

Conclusion

A **Single-Page Application loads the main application once and dynamically changes its content without refreshing the entire page**. In a MERN application, React handles the frontend while Express/Node.js provides APIs and MongoDB stores data. In contrast, a traditional multi-page application requests and loads a new HTML document whenever the user navigates to another page.

2. Explain TypeScript and discuss its importance in Angular application development.

What is TypeScript?

TypeScript is a programming language developed by Microsoft. It is a **superset of JavaScript**, meaning every valid JavaScript program is also valid TypeScript. It adds features such as **static typing, classes, and interfaces** to JavaScript. TypeScript code is compiled into plain JavaScript, which can run in browsers.

Angular is written entirely in TypeScript, and Angular applications are also developed using TypeScript.

Simple Example

JavaScript:

```javascript
let age = 20;
age = "twenty";   // No type restriction
```

TypeScript:

```typescript
let age: number = 20;
age = "twenty";   // Error
```

Here, : number specifies that age can contain only a number. TypeScript detects the error during development/compilation before the application runs in the browser.

Importance of TypeScript in Angular

1. Static Typing

TypeScript allows developers to specify the type of variables, parameters, and return values.

```typescript
let name: string = "Akash";
let age: number = 22;
let isStudent: boolean = true;
```

This helps prevent type-related errors.

---

2. Early Error Detection

TypeScript can identify many errors **before the application runs**.

```TypeScript
let count: number = 0;

count = "oops";  // Type error
```

The error is detected at compile time rather than waiting for the application to run.

---

3. Supports Classes and Object-Oriented Programming

TypeScript provides **classes and interfaces**, making it suitable for developing large Angular applications using object-oriented concepts.

Example:

```TypeScript
class Student {
  name: string;
  age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}
```

---

4. Interfaces

Interfaces define the expected structure of an object.

```typescript
interface Student {
  name: string;
  age: number;
}

let student: Student = {
  name: "Akash",
  age: 22
};
```

This helps maintain consistency in the application's data.

---

5. Better Code Completion and IntelliSense

TypeScript provides **smart auto-completion, type checking, navigation, and refactoring support** in modern code editors. This makes development faster and easier, especially in large projects.

---

6. Supports Angular Decorators

Angular uses TypeScript decorators such as @Component() to define Angular components.

```typescript
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: '<h1>Hello Angular</h1>'
})
export class AppComponent {
  title: string = 'My App';
}
```

Here:

- @Component() → identifies the class as an Angular component.
- AppComponent → TypeScript class.
- title: string → typed property.

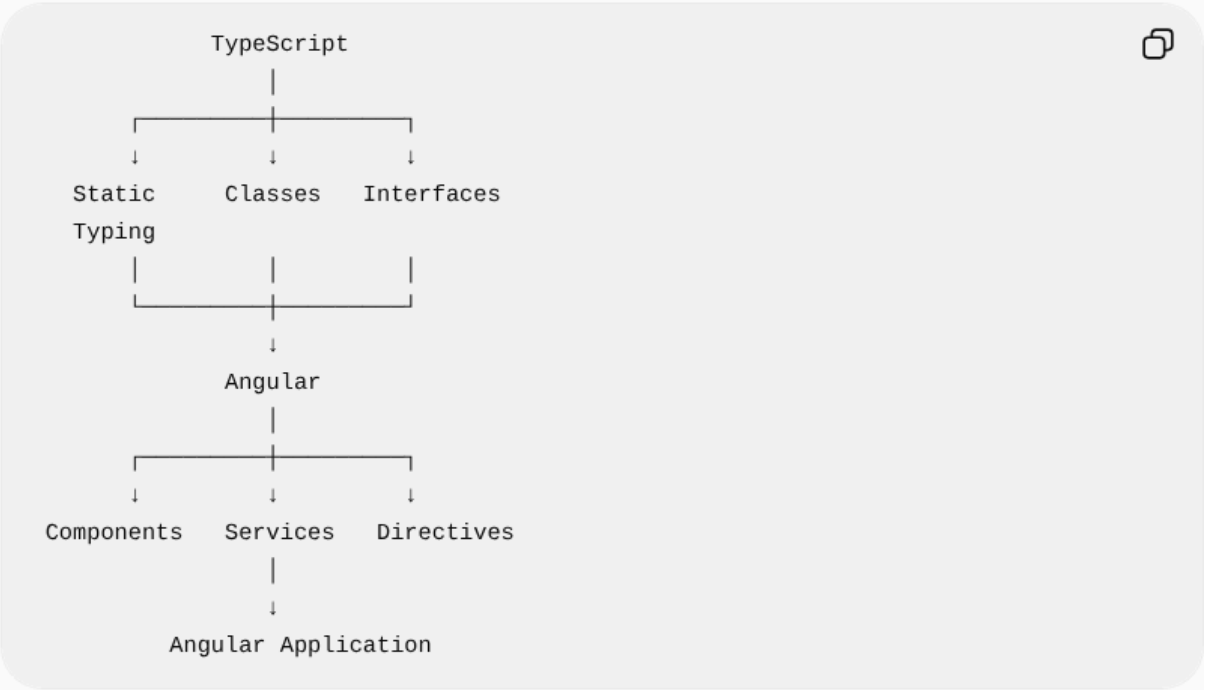
The Angular material specifically identifies @Component as a TypeScript decorator and shows typed component properties.

---

7. Easier Maintenance and Refactoring

In large Angular applications, developers frequently need to change variable names, methods, classes, or interfaces. TypeScript's tooling helps safely perform such changes across the project.

TypeScript and Angular Relationship



Key Benefits

Feature	Importance in Angular
Static typing	Reduces type-related errors
Classes	Supports object-oriented development
Interfaces	Defines data structures
Decorators	Used extensively by Angular

IntelliSense	Improves coding speed
Error detection	Finds many errors early
Refactoring	Makes large applications easier to maintain
JavaScript compatibility	Compiles to normal JavaScript

Conclusion

TypeScript is the primary programming language of Angular. It extends JavaScript with **static typing, classes, interfaces, decorators, and powerful development tools**. These features help Angular developers detect errors early, write structured code, improve productivity, and maintain large applications more easily.

3. Explain the role of components, directives, and pipes in an Angular application.

Angular applications are built using several **core building blocks**, among which **components, directives, and pipes** are important for creating and controlling the user interface.

1. Components

A **component** is a reusable building block of an Angular application. It contains the **logic and view** for a particular part of the application.

A component is generally created using a TypeScript class with the @Component decorator.

Example

TypeScript

```typescript
@Component({
  selector: 'app-hello',
  template: '<h2>Hello {{ name }}</h2>'
})
export class HelloComponent {
  name = 'Akash';
}
```

Here:

- @Component → identifies the class as a component.
- selector → defines the HTML tag used to display it.
- template → defines what is displayed.
- name → stores component data.

For example:

<app-hello></app-hello>

Angular replaces this tag with the component's rendered output.

Role: Components divide an application into **small, reusable, self-contained UI sections**.

---

2. Directives

A **directive** is an Angular feature that **adds behaviour to an HTML element or changes how the element is displayed or works**.

Types of Directives

a) Component Directive

Every Angular component is technically a directive with its own template.

<app-hello></app-hello>

b) Attribute Directive

It changes the **appearance or behaviour** of an existing element without changing its DOM structure.

Examples:

HTML

```
<p [ngClass]="myClass">Hello</p>
```

HTML

```
<p [ngStyle]="myStyle">Hello</p>
```

ngClass and ngStyle are examples of attribute directives.

c) Structural Directive

It changes the **structure of the DOM** by creating, removing, or repeating elements.

Examples:

HTML

```
<p *ngIf="isLoggedIn">Welcome</p>
```

HTML

```
<li *ngFor="let student of students">
  {{ student }}
</li>
```

*ngIf, *ngFor, and *ngSwitch are structural directives.

Role: Directives tell Angular **how an element should behave, appear, or whether it should exist**.

---

3. Pipes

A **pipe** is used to **transform or format data before displaying it in the template**. Pipes use the | symbol and do not modify the original data stored in the component.

Syntax

{{ value | pipeName }}

Example

TypeScript

```typescript
name = "akash";
price = 2500;
```

HTML

```html
<p>{{ name | uppercase }}</p>
<p>{{ price | currency:'INR' }}</p>
```

Output:

AKASH
₹2,500.00

Angular provides several built-in pipes such as:

- uppercase
- lowercase
- date
- currency
- percent
- number
- slice
- json

Role: Pipes make data **more readable and user-friendly** without changing the original value.

Difference Between Components, Directives and Pipes

Feature	Components	Directives	Pipes
Purpose	Create UI sections	Add behaviour/change elements	Transform displayed data
Main use	Build application UI	Control appearance/DOM	Format data
Has template?	Yes	Generally no	No
Example	@Component	*ngIf, ngClass	uppercase, currency

Example use	<app-home>	*ngIf="condition"	{{ name | uppercase }}

Easy Way to Remember

```
COMPONENTS
    ↓
Build the UI

DIRECTIVES
    ↓
Control / change the UI

PIPES
    ↓
Format the data shown in UI
```

Conclusion

Components are the main building blocks used to create Angular application's UI. **Directives** add behaviour or modify how HTML elements are displayed and structured. **Pipes** transform data into a suitable format for presentation. Together, they make Angular applications **modular, reusable, dynamic, and easier to maintain**.

4. Explain Angular component architecture with a suitable example. Discuss how data binding is used between the component and its template.

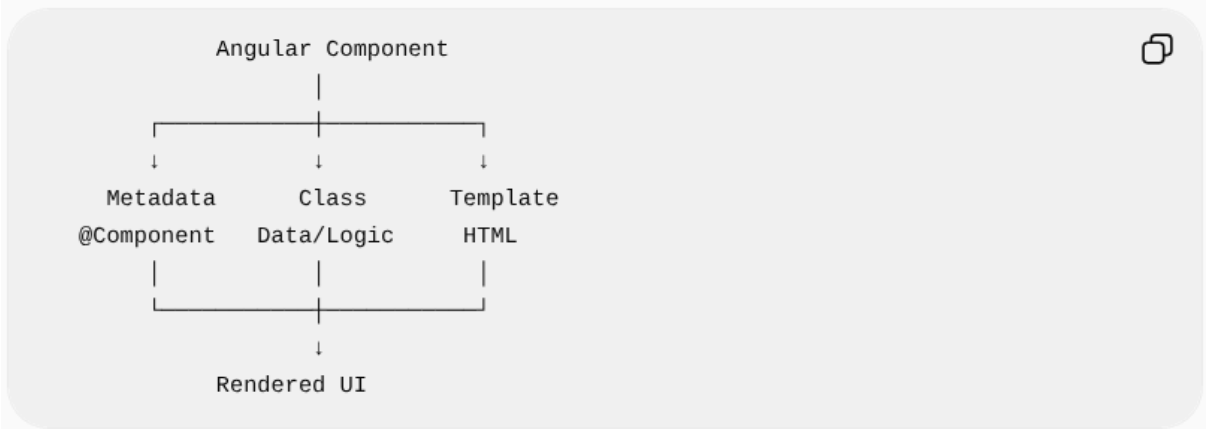
Angular Component Architecture

An **Angular component** is a basic building block of an Angular application. A component controls a specific part of the user interface and contains the **logic, data, and template** required to display that part.

An Angular component mainly consists of:

1. **Component Class** – contains data and application logic.
2. **Template** – defines the HTML/UI.
3. **Metadata** – provided using the @Component decorator.
4. **Styles** – define the appearance of the component.

Basic Architecture



Suitable Example

Component Class

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-student',
  templateUrl: './student.component.html'
})
export class StudentComponent {
  name: string = 'Akash';
  age: number = 22;

  changeName() {
    this.name = 'Rahul';
  }
}
```

Here:

- @Component → defines the class as an Angular component.
- selector → defines the custom HTML element.
- templateUrl → connects the component with its HTML template.
- name and age → component data.
- changeName() → component method.

Template

```
<h2>Student Details</h2>

<p>Name: {{ name }}</p>
<p>Age: {{ age }}</p>
```

<button (click)="changeName()">Change Name</button>

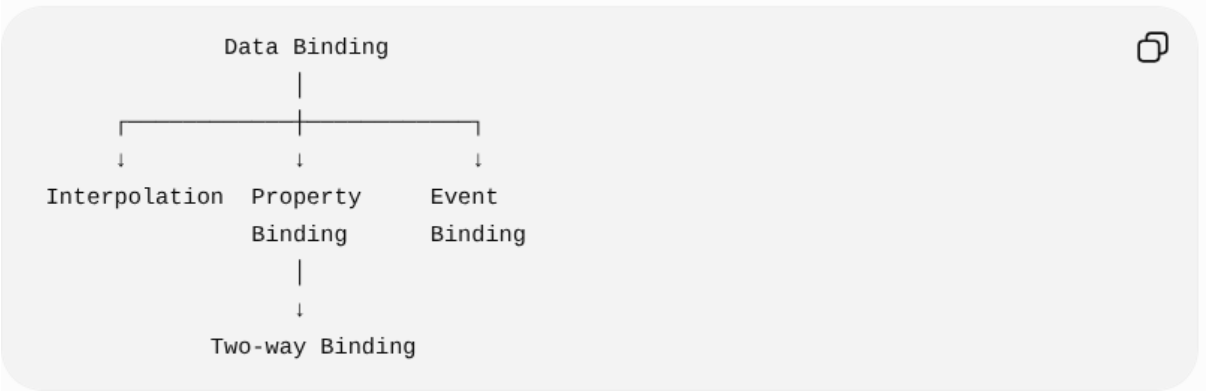
The component class contains the **data and logic**, while the template displays the data and handles user interaction.

Data Binding in Angular

Data binding is the mechanism used to establish communication between the **component class and its template**.

It allows data to move between the TypeScript class and HTML.

There are four main types of data binding:



1. Interpolation

Interpolation is used to display component data in the template.

Syntax:

{{ expression }}

Example:

name = 'Akash';
Hello {{ name }}

Output:

Hello Akash

Data flows:

Component → Template

---

2. Property Binding

Property binding is used to bind a component property to an HTML element property.

Syntax:

[property]="expression"

Example:

imageUrl = 'student.jpg';
<img [src]="imageUrl">

Here, the value of imageUrl from the component is assigned to the image's src property.

Component → HTML Element

---

3. Event Binding

Event binding allows the **template to send information/actions to the component** when a user interacts with the UI.

Syntax:

(event)="method()"

Example:

<button (click)="changeName()">Change Name</button>

When the button is clicked, Angular calls the changeName() method in the component.

Template → Component

---

4. Two-Way Data Binding

Two-way binding allows data to flow **both from component to template and from template back to component**.

It is commonly achieved using [(ngModel)].

Example:

```
name = 'Akash';
<input [(ngModel)]="name">

<p>{{ name }}</p>
```

If the user changes the value in the input box, the name property is also updated.

```
Component
   ↕
Template
```

Summary

Binding	Syntax	Direction	Purpose
Interpolation	{{ name }}	Component → Template	Display data
Property Binding	[src]="imageUrl"	Component → Template	Set element properties
Event Binding	(click)="method()"	Template → Component	Handle events
Two-way Binding	[(ngModel)]="name"	Both directions	Keep data synchronized

Conclusion

Angular component architecture separates **application logic from the user interface**. The **TypeScript component class** manages data and behaviour, while the **template** defines the UI. **Data binding** connects these two parts, allowing data and user actions to move between the component and template efficiently.

5. Explain the steps involved in building your first Angular application.

Angular applications are created and managed using the **Angular CLI (Command-Line Interface)**. The CLI helps to create, develop, build, and maintain Angular applications.

Step 1: Install Prerequisites

First, make sure **Node.js and NPM** are installed.

Then install Angular CLI globally:

npm install -g @angular/cli

Verify the installation:

ng version

The Angular CLI provides the commands required to create and manage Angular projects.

Step 2: Create a New Angular Application

Run:

ng new hello-world-app

The CLI creates the complete project structure, including source files, configuration files, and dependencies.

During creation, Angular may ask some configuration questions, such as:

Add Angular routing? → No
Stylesheet format? → CSS

Step 3: Move Into the Project Folder

After creating the project:

cd hello-world-app

All further Angular CLI commands are normally executed from inside this project folder.

Step 4: Run the Application

Start the development server:

ng serve

or:

ng serve -o

The -o option automatically opens the application in the browser.

The application can be accessed at:

http://localhost:4200

ng serve compiles the project, builds it in memory, starts the development server, and watches for changes.

Step 5: Create a Component

Angular applications are made up of components.

Create a component using:

ng generate component hello-world

Short form:

ng g c hello-world

Angular automatically creates the required files and boilerplate code.

It generates files such as:

```
hello-world/
 ├── hello-world.component.ts
 ├── hello-world.component.html
 ├── hello-world.component.css
 └── hello-world.component.spec.ts
```

The TypeScript file contains the component logic, HTML contains the template, CSS contains styling, and the spec file contains tests.

Step 6: Add Content to the Component

hello-world.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  templateUrl: './hello-world.component.html'
})
export class HelloWorldComponent {
```

}

hello-world.component.html

<h2>Hello World!</h2>
My first Angular application.

The component's selector can then be used in another template:

<app-hello-world></app-hello-world>

Angular renders the component wherever its selector is used.

Overall Process

Install Node.js + NPM
↓
Install Angular CLI
↓
ng new hello-world-app
↓
cd hello-world-app
↓
ng serve -o
↓
Application opens in browser
↓
ng generate component hello-world
↓
Write HTML + TypeScript
↓
Angular renders the application

Important Angular CLI Commands

Command	Purpose
npm install -g @angular/cli	Install Angular CLI
ng version	Check Angular CLI version
ng new app-name	Create a new Angular project
ng serve	Run development server

ng g c component-name	Generate a component
ng build	Build application for production

Conclusion

To build a first Angular application, we **install Angular CLI, create a project using ng new, enter the project directory, run it using ng serve, and create components using Angular CLI commands**. Angular CLI automatically generates much of the required project structure, making Angular application development easier and organized.

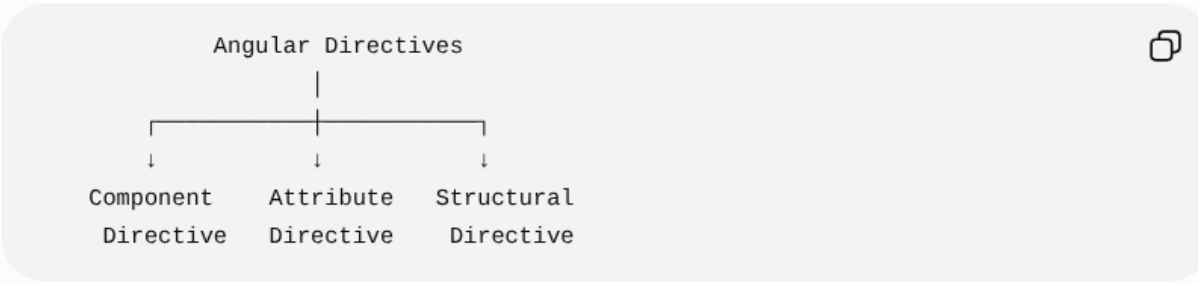
6. What are Angular directives? Explain the different types of directives with suitable examples.

Angular directives are special instructions in Angular that are used to **change the appearance, behaviour, or structure of HTML elements**. They tell Angular how an element should behave or be displayed.

For example, a directive can make an element appear only when a condition is true or change its CSS style.

Types of Angular Directives

Angular directives are mainly classified into **three types**:



---

1. Component Directives

A **component** is technically a special type of directive that has its own **template**.

It is used to create and control a particular part of the user interface.

Example

import { Component } from '@angular/core';

```
@Component({
  selector: 'app-student',
  template: '<h2>Student Details</h2>'
})
export class StudentComponent {
}
```

It can be used in HTML as:

```
<app-student></app-student>
```

Angular renders the component's template wherever the selector is used.

Purpose: To create reusable UI components.

---

2. Attribute Directives

Attribute directives change the **appearance or behaviour of an existing HTML element** without changing its basic structure.

Common examples include:

- ngClass
- ngStyle

Example using ngClass

```
<p [ngClass]="{'highlight': isActive}">
   Welcome to Angular
</p>
```

If isActive is true, the highlight CSS class is applied.

Example using ngStyle

```
<p [ngStyle]="{'color': 'blue'}">
   Hello Angular
</p>
```

The text will be displayed in blue.

Purpose: To change the **appearance or behaviour** of existing elements.

---

3. Structural Directives

Structural directives change the **structure of the DOM** by adding, removing, or repeating elements.

Common examples are:

- *ngIf
- *ngFor
- *ngSwitch

a) *ngIf

It displays an element only when a condition is true.

```
<p *ngIf="isLoggedIn">
   Welcome to the application
</p>
```

If isLoggedIn is true, the paragraph is displayed. If it is false, the element is not rendered.

---

b) *ngFor

It is used to repeat an element for each item in a collection.

```
students = ['Akash', 'Rahul', 'Kiran'];
<ul>
  <li *ngFor="let student of students">
    {{ student }}
  </li>
</ul>
```

Output:

Akash
Rahul
Kiran

---

c) *ngSwitch

It displays different content depending on a value.

```HTML
<div [ngSwitch]="role">

  <p *ngSwitchCase="'admin'">Admin User</p>

  <p *ngSwitchCase="'student'">Student User</p>

  <p *ngSwitchDefault>Unknown User</p>

</div>
```

Difference Between the Types

Type	Purpose	Examples
Component	Creates a UI component with its own template	@Component
Attribute	Changes appearance or behaviour	ngClass, ngStyle
Structural	Changes DOM structure	*ngIf, *ngFor, *ngSwitch

Easy Way to Remember

```
Component
   ↓
Creates UI

Attribute Directive
   ↓
Changes existing UI

Structural Directive
   ↓
Adds / removes / repeats UI
```

Conclusion

Angular directives provide instructions that control how HTML elements behave and appear. They are mainly classified as **component directives, attribute directives, and structural directives**. Components create UI sections, attribute directives modify existing

elements, and structural directives change the DOM structure. Thus, directives make Angular applications **dynamic, reusable, and interactive**.

7. Explain the following with respect to Angular components: Lifecycle hooks

Lifecycle hooks are special methods provided by Angular that allow developers to perform actions at **different stages of a component's life**—from its creation to its destruction.

Component Lifecycle

Component Created
↓
ngOnChanges()
↓
ngOnInit()
↓
ngDoCheck()
↓
ngAfterContentInit()
↓
ngAfterContentChecked()
↓
ngAfterViewInit()
↓
ngAfterViewChecked()
↓
Component Used
↓
ngOnDestroy()
↓
Component Destroyed

Important Lifecycle Hooks

1. ngOnChanges()

Called when an **input property** of a component changes.

```
ngOnChanges() {
  console.log("Input value changed");
}
```

Use: To respond to changes in data received from a parent component.

---

2. ngOnInit()

Called **once after Angular initializes the component**.

```
ngOnInit() {
  console.log("Component initialized");
}
```

Use: Commonly used for:

- Initializing component data
- Calling APIs
- Performing initial setup

---

3. ngDoCheck()

Called during Angular's **change-detection process**.

```
ngDoCheck() {
  console.log("Change detection running");
}
```

Use: To perform custom checking when the developer needs more control over detecting changes.

---

4. ngAfterContentInit()

Called once after Angular has initialized content projected into the component using mechanisms such as <ng-content>.

Use: To perform actions after projected content has been initialized.

---

5. ngAfterContentChecked()

Called after Angular checks the **projected content**.

Use: To respond to changes detected in projected content.

---

6. ngAfterViewInit()

Called once after the component's **view and child views have been initialized**.

```
ngAfterViewInit() {
  console.log("View initialized");
}
```

Use: When an operation requires the component's view to already exist.

---

7. ngAfterViewChecked()

Called after Angular checks the component's **view and child views**.

```
ngAfterViewChecked() {
  console.log("View checked");
}
```

Use: To respond after Angular performs view checking.

---

8. ngOnDestroy()

Called **just before Angular destroys the component**.

```
ngOnDestroy() {
  console.log("Component destroyed");
}
```

Use: To perform cleanup such as:

- Unsubscribing from observables
- Removing event listeners
- Clearing timers
- Releasing resources

Simple Example

TypeScript

```typescript
import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-student',
  template: '<h2>{{ name }}</h2>'
})
export class StudentComponent implements OnInit, OnDestroy {

  name = 'Akash';

  ngOnInit() {
    console.log('Component initialized');
  }

  ngOnDestroy() {
    console.log('Component destroyed');
  }
}
```

Here, ngOnInit() runs when the component is initialized, while ngOnDestroy() runs when the component is about to be removed.

Summary

Lifecycle Hook	When it is called	Main purpose
ngOnChanges()	Input value changes	Respond to input changes
ngOnInit()	Component initialization	Initial setup/API calls
ngDoCheck()	During change detection	Custom change checking
ngAfterContentInit()	Projected content initialized	Work with projected content
ngAfterContentChecked()	Projected content checked	Respond to content checking
ngAfterViewInit()	View initialized	Work with initialized view
ngAfterViewChecked()	View checked	Respond after view checking

ngOnDestroy()	Before component destruction	Cleanup resources

Conclusion

Angular lifecycle hooks provide a way to **execute code at specific stages of a component's lifecycle**. They are especially useful for initialization, responding to changes, interacting with views, and cleaning up resources before a component is destroyed.

8. Explain Dependency Injection (DI) in Angular. Describe how dependency injection helps in developing maintainable applications.

Dependency Injection (DI) is a design pattern in Angular where a class **receives the objects or services it needs from Angular instead of creating them itself**.

A **dependency** is an object that another class needs to perform its work. For example, a component may need a service to retrieve data from a server.

How Dependency Injection Works

Angular has a **Dependency Injection system** that creates and provides required services to components and other classes.

```
Component
   |
   | needs
   ↓
Service
   ↑
   | provided by
   |
Angular Injector
```

Instead of a component creating a service manually:

const service = new StudentService();

Angular provides the required service through the constructor:

constructor(private studentService: StudentService) {}

Angular's injector creates/provides the StudentService object and supplies it to the component.

Example

1. Create a Service

TypeScript

```typescript
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class StudentService {

  getStudents() {
    return ['Akash', 'Rahul', 'Kiran'];
  }
}
```

@Injectable() tells Angular that the class can participate in dependency injection.

providedIn: 'root' makes the service available throughout the application.

---

2. Inject the Service into a Component

TypeScript

```typescript
import { Component } from '@angular/core';
import { StudentService } from './student.service';

@Component({
  selector: 'app-student',
  template: `
    <p *ngFor="let student of students">
      {{ student }}
    </p>
  `
})
export class StudentComponent {

  students: string[];

  constructor(private studentService: StudentService) {
    this.students = this.studentService.getStudents();
  }
}
```

What happens here?

```
StudentComponent
       |
       | requests StudentService
       ↓
Angular Injector
       |
       ↓
Creates/provides StudentService
       |
       ↓
StudentComponent receives it
       |
       ↓
Calls getStudents()
```

The component **does not create the service itself**. Angular manages the dependency.

How DI Helps in Developing Maintainable Applications

1. Loose Coupling

DI reduces the dependency between classes.

The component does not need to know **how the service is created**. It only uses the service.

Without DI:
Component → creates Service

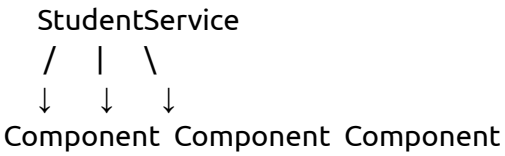
With DI:
Component → requests Service ← Angular Injector

This makes the application easier to modify.

---

2. Code Reusability

A single service can be injected into multiple components.



This avoids writing the same logic repeatedly.

---

3. Easier Testing

Dependencies can be replaced with **mock objects** during testing.

For example, instead of connecting to a real database or API, a mock service can be provided.

This makes unit testing easier and faster.

---

4. Separation of Responsibilities

DI encourages developers to separate application responsibilities.

For example:

Component → UI and user interaction
Service → Business/data logic
Injector → Provides dependencies

Therefore, components remain simpler and easier to understand.

---

5. Easier Maintenance

If the implementation of a service changes, components using that service generally do not need to change.

For example:

Old:
StudentService → Local data

Later:
StudentService → REST API

The component can continue calling:

studentService.getStudents();

without needing to know how the data is obtained.

Advantages of Dependency Injection

- **Loose coupling** between classes
- **Code reusability**
- **Better separation of concerns**
- **Easier unit testing**
- **Simpler maintenance**
- **Centralized dependency management**
- Makes large applications easier to organize

Conclusion

Dependency Injection is an important feature of Angular that allows components and other classes to receive their required services from Angular's injector rather than creating them manually. It promotes **loose coupling, reusability, separation of responsibilities, easier testing, and maintainability**, making it particularly useful for developing large Angular applications.

9. Explain dependency injection through the constructor and the inject() property.

In Angular, **Dependency Injection (DI)** allows a class to receive the services or objects it needs from Angular's **Injector**, instead of creating those objects manually.

There are two common ways to inject a dependency:

1. **Constructor Injection**
2. **inject() Function**

---

1. Dependency Injection Through Constructor

In **constructor injection**, the required service is declared as a parameter of the component's constructor.

Example

First, create a service:

TypeScript

```typescript
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class StudentService {

  getName() {
    return 'Akash';
  }
}
```

Then inject it into a component:

TypeScript

```typescript
import { Component } from '@angular/core';
import { StudentService } from './student.service';

@Component({
  selector: 'app-student',
  template: `<h2>{{ name }}</h2>`
})
export class StudentComponent {

  name: string;

  constructor(private studentService: StudentService) {
    this.name = this.studentService.getName();
  }
}
```

How it works

```
StudentComponent
       |
       | requests StudentService
       ↓
Angular Injector
       |
       ↓
StudentService object
       |
       ↓
Constructor receives it
```

Here:

constructor(private studentService: StudentService)

means that Angular should provide an instance of StudentService to the component.

Advantages

- Simple and easy to understand
- Dependencies are clearly visible in the constructor
- Commonly used in Angular applications
- Makes dependencies available throughout the class

---

2. Dependency Injection Using inject()

Angular also provides the **inject() function** to obtain a dependency directly.

Instead of putting the service in the constructor, we can write:

TypeScript

```typescript
import { Component, inject } from '@angular/core';
import { StudentService } from './student.service';

@Component({
  selector: 'app-student',
  template: `<h2>{{ name }}</h2>`
})
export class StudentComponent {

  private studentService = inject(StudentService);

  name = this.studentService.getName();
}
```

Here:

private studentService = inject(StudentService);

asks Angular's injector to provide an instance of StudentService.

How it works

```
StudentComponent
       |
       | inject(StudentService)
       ↓
Angular Injector
       |
       ↓
StudentService object
```

Constructor Injection vs inject()

Feature	Constructor Injection	inject()

Syntax	constructor(private service: Service)	service = inject(Service)
Dependency location	Constructor	Class field/property
Readability	Very clear dependencies	More compact
Common use	Traditional Angular approach	Modern Angular approach
Requires constructor	Yes	No

Simple Comparison

Constructor:

constructor(private studentService: StudentService) {}

Using inject():

studentService = inject(StudentService);

Both approaches obtain the dependency from **Angular's Dependency Injection system**. The main difference is **where and how the dependency is requested**.

Conclusion

Angular provides both **constructor injection** and the **inject() function** for dependency injection. Constructor injection receives dependencies through the constructor, while inject() obtains them directly within an appropriate injection context, such as a class field. Both approaches help achieve **loose coupling, reusability, easier testing, and maintainable Angular applications**.